

EJEMPLO:

Vamos a realizar un ejemplo aplicado con el cual podamos dar unos primeros pasos en el tema de la programación orientada a objetos y el lenguaje de programación Java.

Lo que vamos a hacer es modelar una caja de seguridad.

¿Qué es una caja de seguridad?

Es un espacio en el que guardamos objetos de valor, es difícil acceder al contenido de esta por varias razones, como el grosor de las paredes, los materiales con los que está construida y por qué el acceso normal requiere de una llave o clave de acceso.



La Abstracción

Empecemos con la primera de las propiedades: la abstracción.

- ¿Cuáles son las características más importantes de la caja?

Podemos decir: el peso, el color, etc. y estaría bien, sin embargo, en este ejemplo vamos a tomar solamente los atributos necesarios para el funcionamiento de la misma. En este caso serían 2: *La clave de acceso* y *el espacio disponible para guardar cosas*.

La clave puede ser cualquier cosa, pero generalmente es una combinación numérica.

El espacio es algo más ambiguo, la mejor forma de imaginar un espacio así es pensar en una cuadrícula, cada objeto que queremos guardar ocupará un cuadro, puede que haya objetos que ocupen más de 1 cuadro, pero la abstracción más simple es que todos los objetos ocupen solamente 1 cuadro.

Entonces ¿no será necesario también saber qué objetos hemos guardado? En efecto, no es necesario solo saber cuánto espacio hay, sino que hay en cada espacio. Esto es importante porque la caja es un contenedor. Entonces agregamos una 3er atributo: Los objetos contenidos en la caja.

Ahora, pasemos a la parte de **los métodos**.

¿Qué podemos hacer con la caja de seguridad?

Primero que nada, es un lugar para guardar cosas, entonces tiene que tener dos funciones: guardar objetos y sacar objetos.

Sin embargo, recordemos que la idea de la caja es que los objetos no se puedan sacar así no más, primero tenemos que obtener acceso a la caja de seguridad, y eso se hace con la clave de acceso, entonces podemos agregar otro método: abrir caja de seguridad y claro, cuando terminemos de guardar o sacar los objetos tenemos que cerrar la caja, por lo que agregaremos un último método: cerrar caja.

Abrir y cerrar la caja tienen que ser la primera y la última acción que se realice siempre que hagamos uso de la caja de seguridad.

Como tenemos que saber si la caja está abierta para poder guardar y sacar objetos tenemos que agregar otra propiedad a nuestra caja, el estado de la puerta que puede ser solamente abierta y cerrada.

Esa sería la definición de la caja, ahora convirtámoslo a una clase en Java, se vería de la siguiente manera.

```
public class Caja{
    private Integer clave;
    private Integer capacidad;
    private Integer contenido;
    private Boolean abierto;

    public Caja (Integer clave_original){
        this.clave = clave_original;
        this.abierto = false;
        this.contenido = 0;
    }

    public Boolean abrirCaja(Integer clave_ingresada){

    }

    public void cerrarCaja(){

    }

    public void guardarObjeto(){

    }

    public void sacarObjeto(){

    }

}
```

Hablemos de lo que tenemos aquí.

- Primero definimos todos los atributos que requiere la clase, como habíamos dicho, una clave, el contenido de la caja y si está abierta o cerrada. Por el momento vamos a poner el contenido como un número, solo para saber la cantidad de objetos que tenemos, más adelante vamos a actualizarlo para mostrar los objetos.
- Luego tenemos un método, este es especial, es el constructor de la clase, en él es donde se crea una instancia de la clase, puede o no requerir parámetros, en este caso va a tomar la clave que le queremos colocar.
- Luego de eso tenemos los demás métodos: abrir y cerrar caja, guardar y sacar un objeto de la caja. Estos los completaremos más adelante.

Vamos a probar la clase, en la misma colocamos una función que se llama main

```
public static void main(String args[]){
    Caja caja1 = new Caja(1234);

    Boolean abierto = caja1.abrirCaja(1234);

    if(abierto){
        System.out.println("Caja abierta");
    }else{
        System.out.println("Caja aun cerrada");
    }
}
```

No nos preocupemos de cómo está definida la función por el momento vamos a copiarla nada más.

Lo que hace es crear una instancia de la caja, le coloca una clave y luego revisa si está abierta o no. Si la clave es correcta, el mensaje nos va a decir "Caja abierta", si la clave no es correcta nos dirá: "Caja aún cerrada"

¿Cómo hace esto? pues tenemos que modificar los métodos de abrir y cerrar caja para que puedan cambiar el estado de la caja de esta forma.

```
public Boolean abrirCaja(Integer clave_ingresada){
    if(this.clave.equals(clave_ingresada)){
        this.abierto = true;
    }else{
        this.abierto = false;
    }
    return this.abierto;
}

public void cerrarCaja(){
    this.abierto = false;
}
```

El método de cerrar caja es simple, solo cambiamos el atributo de abierto a falso, el de abrir caja tiene un poco de lógica, primero tiene que revisar si la clave ingresada es correcta, de no serlo se vuelve a cerrar la caja.

La clase completa se verá de esta forma:

```
public class Caja{
    private Integer clave;
    private Integer capacidad;
    private Integer contenido;
    private Boolean abierto;

    public Caja (Integer clave_original){
        this.clave = clave_original;
        this.abierto = false;
    }

    public Boolean abrirCaja(Integer clave_ingresada){
        if(this.clave.equals(clave_ingresada)){
            this.abierto = true;
        }else{
            this.abierto = false;
        }
    }
}
```

```
        }

        return this.abierto;

    }

    public void cerrarCaja(){
        this.abierto = false;
    }

    public void guardarObjeto(){

    }

    public void sacarObjeto(){

    }

    public static void main(String args[]){
        Caja caja1 = new Caja(1234);

        Boolean abierto = caja1.abrirCaja(1234);

        if(abierto){
            System.out.println("Caja abierta");
        }else{
            System.out.println("Caja aun cerrada");
        }
    }
}
```

Si ejecutamos ese archivo nos va a decir que la caja está abierta, si cambiamos el valor de `Boolean abierto = caja1.abrirCaja(1234);` por cualquier otro número nos va a decir que la caja está cerrada.

Como último punto consideremos el uso del método `equals()`. ¿Por qué no solo comparar con el doble igual `==` ? En Java `==` compara las referencias de los objetos, dicho de manera sencilla, pensemos que dos objetos son dos hojas de papel, cada una tiene el número 2 escrito, si preguntamos si el contenido es el mismo es verdadero. El problema es que `==` no pregunta que tienen escritas las

dos hojas, pregunta si es la misma hoja. El método equals solamente compara el contenido.

El encapsulamiento

Ahora vamos a la siguiente propiedad de la programación: El encapsulamiento.

Cuando hablamos de programación hay un gran problema ¿quién puede hacer cambios en el estado de un objeto? Para ilustrar esto sigamos con el ejemplo de la caja de seguridad. ¿Quién decide si la caja está abierta o cerrada? La respuesta es: la caja misma, o más bien, el mecanismo de seguridad de la caja. Si se usa la clave correcta, la caja permite que se abra la puerta, pero mientras la clave no sea la correcta, no se puede cambiar el estado de la caja.

En programación orientada a objetos la puerta de la caja sería una variable privada. Nadie más que la caja misma puede cambiar el estado de la puerta. Solo la caja puede hacerlo dependiendo de si le damos la clave correcta o no.

El nivel de acceso de las propiedades de un objeto es un tema delicado, público o privado no significa solamente quién puede ver una propiedad, sino también quién puede modificarla. En esta clase está bien si usamos campos públicos por la facilidad de uso de estos así:

```
public class Caja{  
    public Integer clave;  
    public Integer capacidad;  
    public Integer contenido;  
    public Boolean abierto;  
  
    ...  
}
```

Sin embargo, una mejor práctica es que los objetos hagan uso de los campos privados y los llamados “métodos de acceso” que son métodos que se encargan de manejar el estado de las propiedades. La ventaja de hacer uso de los métodos de acceso es que podemos validar el estado y si nos dan un valor, por ejemplo, para abrir la puerta, podemos verificar que sea el valor correcto. Eso ya lo trabajamos anteriormente, la puerta de la caja solo se abre si la clave ingresada es la correcta. Si estuviéramos trabajando con atributos públicos y alguien simplemente decide cambiar abierto a verdadero no hay forma de verificar o detener a dicha persona. Al hacer uso de los métodos de acceso podemos controlar la lógica de cómo y cuándo queremos que se abra la puerta.

Ya con eso vamos a agregar un poco de funcionalidad a los métodos de guardar y sacar objetos. Como tenemos el contenido simplemente con la cantidad vamos a colocar lo siguiente.

```
public void guardarObjeto(){  
    if(this.abierto){  
        this.contenido++;  
        System.out.println("Hay "+ this.contenido+" objetos en la caja")  
    }  
}
```

```
public void sacarObjeto(){
    if(this.abierto){
        this.contenido--;
        System.out.println("Hay "+ this.contenido+" objetos en la caja")
    }
}
```

Aquí simplemente verificamos si la caja está abierta, de ser así podemos sacar o guardar objetos, de no estar abierta no podemos hacer nada. Vamos a cambiar luego el funcionamiento para guardar objetos y poder ver lo que tenemos guardado.

Podemos probarlo de la siguiente manera modificando el main:

```
public static void main(String args[]){
    Caja caja1 = new Caja(1234);

    Boolean abierto = caja1.abrirCaja(1234);

    if(abierto){
        System.out.println("Caja abierta");
    }else{
        System.out.println("Caja aun cerrada");
    }

    caja1.guardarObjeto();
    caja1.sacarObjeto();
}
```

Si colocamos múltiples veces las últimas líneas podemos ver como la cantidad de objetos aumenta o disminuye.

Hay un último problema y es que si no hay objetos no se pueden sacar ¿correcto? La solución es simplemente tener otra verificación en el método de sacar Objeto para ver que la cantidad de objetos sea mayor que 1.

La Herencia

Veamos un ejemplo de la próxima propiedad: la herencia. Sin embargo, no vamos a hablar de la caja en este caso, sino de lo que vamos a guardar en ella: Los objetos de valor.

¿Qué es un objeto de valor? Puede ser cualquier cosa: un título de propiedad, joyas, dinero, objetos con valor sentimental. Vamos a hablar solamente de 1 tipo para este ejemplo: las joyas.

¿Por qué las joyas? Simple, sabemos que una joya es un accesorio, hecho de algún metal precioso y generalmente adornado con una gema. Sin embargo, como es un accesorio hay varios tipos de joyas y el tipo no es solamente una propiedad o atributo, sino que cambia totalmente como se usa, como se cuida, como se repara.

Algunos ejemplos de joyas son: aretes, collares y brazaletes.

Podemos hacer una clase para cada uno de estos, sin embargo, veremos que todos tendrán ciertas características similares, como dijimos anteriormente: están hechas de algún metal precioso y pueden tener alguna gema.

La clase Joya se vería de esta forma

```
public class Joya{
    protected String metal;
    protected String gema;

    public Joya(String metal, String gema){
        this.metal = metal;
        this.gema = gema;
    }

    public void limpiar(){
        System.out.println("Limpiando la joya");
    }
    public String toString(){
        return "Joya "+this.metal+" con "+this.gema;
    }
}
```

Veamos el último método que tenemos ahí.

Ese método nos va a devolver un String, que nos dice de qué tipo de metal está hecha la Joya y que gema tiene. Si no colocamos este método en nuestras clases de Java, cuando queramos mostrar la información de la clase Java solamente nos va a decir si existe o no, mostrándonos algo como Joya@27103.

Vamos a utilizar un arreglo para guardar la lista de Joyas, sin embargo, no vamos a usar el arreglo normal, vamos a hacer uso de una librería. En Java, para mostrar que vamos a utilizar librerías, hay que colocarlos hasta arriba del archivo, de la siguiente manera.

```
import java.util.ArrayList;
```

Luego vamos a modificar un poco la clase, se debería ver así:

```
import java.util.ArrayList;

public class Caja {
    private Integer clave;
    private Integer capacidad;
    private Boolean abierto;
    private ArrayList<Joya> contenido;

    public Caja ( Integer clave_original, Integer capacidad_total ) {
        this.clave = clave_original;
        this.capacidad = capacidad_total;
        this.abierto = false;
        this.contenido = new ArrayList<Joya>();
    }
    ...
}
```

Aquí lo que hacemos es inicializar el espacio en el que vamos a guardar las Joyas.

Ahora vamos a modificar los métodos de guardarObjeto y sacarObjeto para que guarden las Joyas que le mandamos.

El primero, guardarObjeto se verá de esta forma

```
public void guardarObjeto (Joya joya){
    if(this.abierta){
        if(this.contenido.size() < this.capacidad){
            //this.contenido += 1;
            this.contenido.add(joya);
            System.out.println("Objeto guardado");
            System.out.println("Objetos en la caja "+this.contenido);
        }
    }
}
```

```
        }else{
            System.out.println("La caja está llena, no se puede guardar");
        }
    }else{
        System.out.println("Caja cerrada, no se puede guardar");
    }
}
```

Veamos parte por parte los cambios que realizamos

- Primero que nada, cambiamos la firma del método, es decir la definición, antes era un método que no recibía parámetros, ahora, recibe un parámetro de tipo Joya, que es la Joya que vamos a guardar en la caja.
- Podemos ver la misma comprobación para verificar que la caja está abierta, eso no cambió del método anterior.
- Luego verificamos que no hayamos excedido el límite de objetos que podemos guardar. Si hay espacio, es decir, si no hemos alcanzado la capacidad de la caja, guardamos el objeto.
- En caso de que no haya espacio en la caja, vamos a mostrar un error.

Ahora cambiemos el método sacarObjeto de esta forma

```
public void sacarObjeto (Joya joya){
    if(this.abierta){
        //this.contenido -= 1;

        for(int i = 0; i < this.contenido.size(); i++){
            if(contenido.get(i).equals(joya)){
                contenido.remove(i);
                System.out.println("Objeto retirado");
                System.out.println("Objetos en la caja "+this.contenido);
                return;
            }
        }
    }else{
        System.out.println("Caja cerrada, no se puede retirar");
    }
}
```

Podemos ver que es muy similar a guardarObjeto, eso es porque sigue mucha de la misma lógica. Revisar que la caja esté abierta, revisar el contenido de los espacios, etc. Veamos entonces cómo es que se sabe cuál objeto sacar.

- Tenemos un simple If, que hace uso de equals, y si resulta que los objetos son iguales, nos va a permitir sacarlos. Esto puede dar algunos problemas, pero resolveremos más adelante.

Ahora que tenemos la clase base Joya, vamos a ver la herencia, lo vamos a hacer creando clases específicas de Joya, como la clase Brazalete

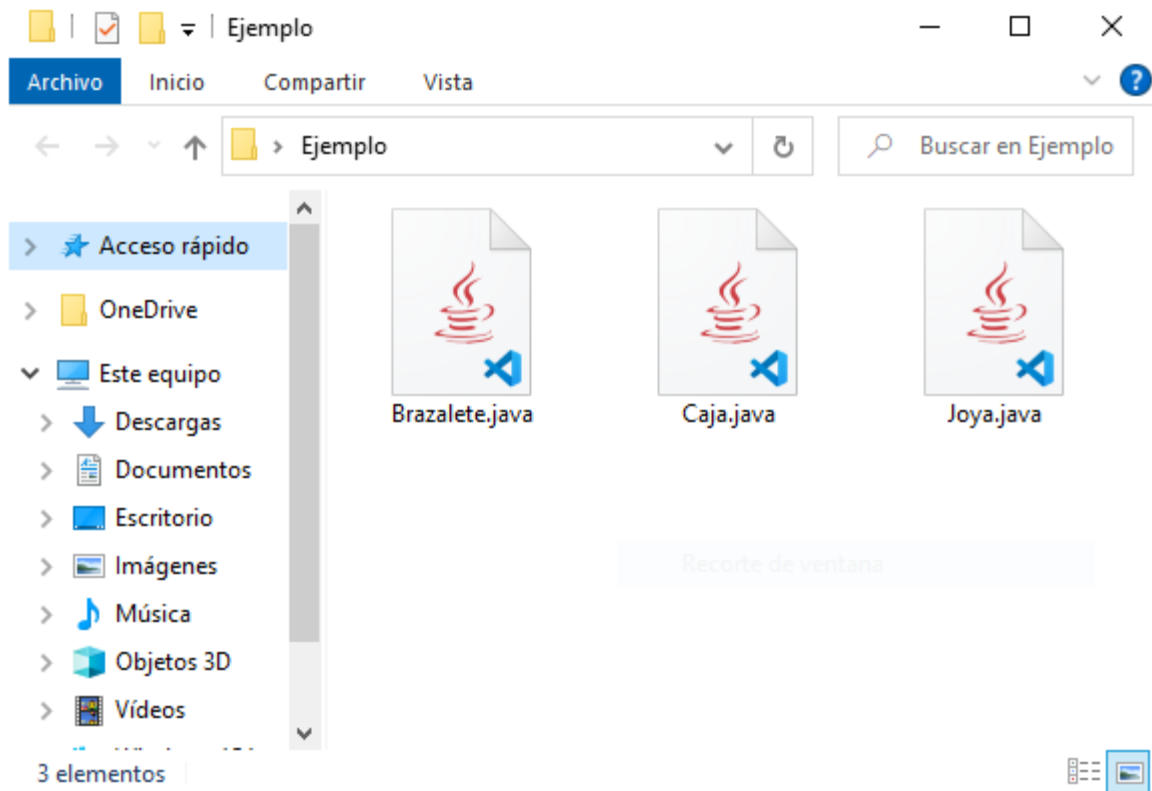
```
public class Brazalete extends Joya {  
    public Brazalete(String metal, String gema){  
        this.metal = metal;  
        this.gema = gema;  
    }  
  
    public void limpiar(){  
        System.out.println("Limpiando el brazalete de "+this.metal);  
    }  
}
```

Esta será la clase, lo único que hacemos es cambiar el método de limpiar para demostrar que es una clase diferente. Pero aquí tenemos un problema, nuestros editores de texto nos dirán que las propiedades metal y gema no son visibles.

Esto ocurre porque son privadas, privado significa que nadie puede verlas, ni siquiera las clases "hijas" la solución es cambiar las propiedades a

protected, eso permite que las clases que heredan puedan acceder a esas propiedades.

Las clases que vamos a crear, las vamos a guardar en una carpeta junto a nuestro archivo de Caja.java, el contenido de la carpeta se vería así.



Lo hacemos de esta manera porque así podemos hacer uso de las distintas clases en un solo programa. Si no estuvieran en la misma carpeta tendríamos que importarlas de otra forma y eso es algo un poco más avanzado.

Volviendo al ejemplo, vamos al archivo de caja, al método main, y vamos a crear un par de objetos.

```
public static void main(String args[]){  
    ...  
    Joya joya1 = new Joya();  
    Brazaletes brazaletes1 = new Brazaletes("plata", "rubi");  
}
```

```
joya1.limpiar();  
brazalete1.limpiar();  
}
```

Si corremos ahora el archivo Caja, veremos que se muestran correctamente los mensajes de Joya y de Brazalete.

Ahora para que veamos cómo funciona la herencia, vamos al archivo Brazalete y borremos el método de limpiar.

Si volvemos a ejecutar el archivo Caja, veremos que ahora tenemos dos líneas con el mismo mensaje.

Esto se conoce como sobrescribir un método, cuando una clase hereda de otra puede cambiar el comportamiento de los métodos que hereda, sin embargo, no es necesario que lo haga, en ese caso puede hacer uso del comportamiento que tiene ya definida la clase padre.

Para completar esta parte vamos a hacer otras dos clases: Arete y Collar, similares a la clase Brazalete y que también sobrescriban el método limpiar. Les queda como tarea completar este ejercicio.

El polimorfismo

Finalmente tratemos la última propiedad, el polimorfismo. Este se refiere a que cuando hagamos uso de la herencia para definir objetos, cuando necesitemos un objeto "Padre" podemos usar un objeto "Hijo".

Ya teniendo las clases hijas, a la hora de querer usarlas hacemos algo como esto

```
Joya joya1 = new Joya();  
Brazalete brazalete1 = new Brazalete("plata", "rubi");  
Arete arete1 = new Arete("oro", "diamante");  
Collar collar1 = new Collar("plata", "esmeralda");
```

Pero también es posible hacer algo como esto

```
Joya joya1 = new Joya();  
Joya brazalete1 = new Brazalete("plata", "rubi");  
Joya arete1 = new Arete("oro", "diamante");  
Joya collar1 = new Collar("plata", "esmeralda");
```

¿Por qué? Porque todas las clases que heredan de Joya son Joyas, eso significa que si alguien nos pide una Joya podemos darle cualquiera de las 3, collares, brazaletes o aretes.

Para verlo aplicado retomemos nuevamente el ejemplo de la caja de seguridad. Ahora si vamos a cambiar el contenido de la caja guardando los objetos que nos dan.

Ya tenemos la caja y tenemos 3 tipos de Joyas: Aretes, Collares y Brazaletes. Vamos a guardar las Joyas haciendo uso del polimorfismo, es decir en lugar de guardarlo así:

```
private Brazaletes[] brazaletes;  
private Arete[] aretes;  
private Collar[] collares;
```

Podemos guardarlos con polimorfismo de esta forma

```
private Joya[] contenido;
```

así todas las Joyas son válidas y no tenemos que estar llevando cuenta de donde guardarlas, así que cambiaremos el contenido de Integer a un arreglo de tipo Joya.

En este ejemplo vamos a utilizar una librería de Java que se llama ArrayList, esta librería permite manejar arreglos de manera más sencilla. Es posible realizar todo lo que vamos a hacer aquí con un arreglo sencillo.

Ahora vamos a guardar algunos objetos, coloquemos lo siguiente en el main.

```
Joya joya1 = new Joya("oro", "diamante");  
Brazaletes brazaletes1 = new Brazaletes("plata", "rubí");  
Arete arete1 = new Arete("oro", "diamante");  
Collar collar1 = new Collar("plata", "esmeralda");  
  
caja1.guardarObjeto(joya1);  
caja1.guardarObjeto(brazaletes1);  
caja1.guardarObjeto(arete1);  
caja1.sacarObjeto(brazaletes1);  
caja1.guardarObjeto(collar1);
```

Finalmente, hay otro problemita más, para sacar la Joya de la caja, estamos usando el mismo objeto que ya habíamos creado, pero en muchos

casos no vamos a tener el objeto original y hacer algo como esto nos va a dar error.

```
Brazalete brazalete1 = new Brazalete("plata", "rubi");  
Brazalete brazalete2 = new Brazalete("plata", "rubi");  
  
caja1.guardarObjeto(joya1);  
caja1.guardarObjeto(brazalete1);  
caja1.guardarObjeto(arete1);  
caja1.sacarObjeto(brazalete2);
```

Verlo así tiene sentido, el brazalete2 no es el que se guardó, pero si, por ejemplo, alguien más fue el que guardó el brazalete, es posible que no tengamos ese objeto, y tenemos que crear otro que cumpla la misma descripción para poder sacar el objeto que queremos.

Si vemos el código de sacarObjeto, vemos que comparamos los objetos por medio de equals, este método, al igual que toString, está disponible para todos los objetos, y también podemos modificar su comportamiento. Entonces vamos a hacer lo siguiente en la clase Joya.

```
public Boolean equals(Joya joya){  
    Boolean tipoMetal = this.metal.equals(joya.metal);  
    Boolean tipoGema = this.gema.equals(joya.gema);  
  
    Boolean igual = tipoJoya && tipoMetal && tipoGema;  
  
    return igual;  
}
```

Aquí estamos haciendo una comparación, para ver si 2 joyas son iguales.
¿Cuáles 2 joyas si solamente recibimos un parámetro?

Resulta que, en Java, como todos los métodos están asociados a un objeto, existe una instancia del mismo, a la cual nos referimos con la palabra reservada *this*.

El método equals, se llama así: `joya1.equals( joya2 )` entonces, aquí tenemos dos joyas: `joya2` es el parámetro al cual nos referimos por nombre, y `joya1` es la instancia del objeto que quiere hacer la comparación, a la cual nos referimos con *this*.

Pero ahora hay un problema, si tenemos varios tipos que pueden cumplir la clase Joya (brazalete, collar y arete) entonces: ¿No nos diría que es lo mismo un collar y un arete si tienen los mismos atributos? Si, eso es correcto. Nos diría que es lo mismo.

Entonces, no solamente tenemos que comparar los atributos, sino también el tipo de objeto que es. Lo que vamos a hacer es exactamente eso, agregando una línea de comparación.

```
public Boolean equals(Joya joya){
    Boolean tipoJoya = (this.getClass().equals(joya.getClass()));
    Boolean tipoMetal = this.metal.equals(joya.metal);
    Boolean tipoGema = this.gema.equals(joya.gema);

    Boolean igual = tipoJoya && tipoMetal && tipoGema;

    return igual;
}
```

La primera línea nos permite comparar que sean del tipo correcto.

Finalmente, modifiquemos el main una última vez y coloquemos esto

```
Caja caja1 = new Caja(1234,3);
caja1.abrirCaja(1234);
Joya joya1 = new Joya("oro", "diamante");
Brazalete brazalete1 = new Brazalete("plata", "rubi");
Brazalete brazalete2 = new Brazalete("plata", "rubi");
Arete arete1 = new Arete("oro", "diamante");
Collar collar1 = new Collar("plata","esmeralda");

caja1.guardarObjeto(joya1);
caja1.guardarObjeto(brazalete1);
caja1.guardarObjeto(arete1);
caja1.verContenido();
caja1.sacarObjeto(brazalete2);
caja1.verContenido();
caja1.guardarObjeto(collar1);
caja1.verContenido();
```

Aquí vamos a ver cómo se comporta la caja, guardamos 3 joyas, luego vemos el contenido, podemos sacarlas y guardar nuevas. Si colocamos la penúltima línea antes de sacar el brazalete, nos va a dar error, de espacio insuficiente. Asimismo, si tratamos de sacarObjeto(collar1), antes de guardarlo, nos va a dar error, que no se pudo encontrar.